

ADAPTIVEVEC: MASTER ENGINEERING & RESEARCH HANDBOOK

A Complete Ground-Up Guide to Theory, Algorithms, Empirical Results, and End-Sem Roadmap for Team Members

Repository & System: AdaptiveVec Proximity Graph Engine (C++20 AVX2 Core + Python API + React 19 Telemetry)

Target Standard: IEEE/ACM Transactions on Knowledge & Data Engineering (TKDE) Camera-Ready Standard

Evaluated Corpora: Texmex SIFT-100K (128d), Stanford GloVe-100K (100d), Fashion-MNIST (784d), Synthetic (64d)

Primary Deliverable: Standalone Technical Dossier for Project Defense, Viva Voce, and Engineering Onboarding

HOW TO USE THIS HANDBOOK (FOR TEAMMATES)

This document assumes **zero prior knowledge** of vector search, graph theory, or quantization. It explains why industry engines (FAISS, Milvus) break on real-world data, the exact mathematical formulas and C++ SIMD optimizations we built, our verified empirical numbers (+50.3% speedup, -62.4% RAM reduction), and exactly what tasks remain for Phase 2 (End-Sem).

Chapter 1: Vector Search from Scratch (The Absolute Basics)

1.1 What is an Embedding Vector?

In modern artificial intelligence (ChatGPT, Google Lens, Spotify recommendations), data like words, photos, and audio cannot be searched using standard SQL keyword matching (e.g., `SELECT * WHERE text = 'cat'`). Instead, neural networks convert raw data into an **embedding vector**—a list of floating-point numbers in a high-dimensional space (e.g., 128 numbers for SIFT image features, 768 numbers for BERT text models, or 1,536 for OpenAI embeddings). Vectors that have similar semantic meaning sit physically close together in this space.

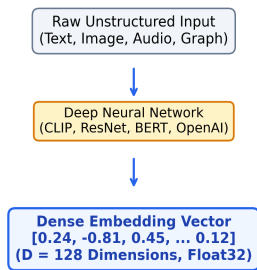
1.2 The Brute-Force Scaling Wall: Why Linear Scan Dies

When a user submits a query (e.g., searching for a similar image), the computer must find the k nearest neighbors (k-NN) by measuring Euclidean distance: $dist(u, v) = \sqrt{\sum (u_i - v_i)^2}$. If you have **1,000,000 vectors** of 128 dimensions, every single query requires 128,000,000 floating-point operations. If 1,000 users query the database every second, the CPU must perform **128 billion calculations per second**. Linear brute-force search is mathematically exact (100% recall), but computationally impossible at web scale.

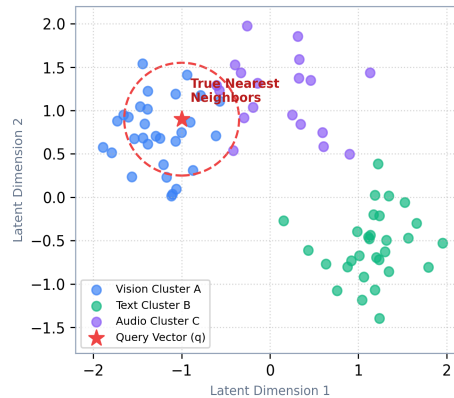
1.3 Approximate Nearest Neighbor (ANN) & Proximity Graphs

To solve this, the computer science community uses **Approximate Nearest Neighbor (ANN)** search. We accept a tiny margin of error (e.g., finding the true nearest neighbor 98% of the time instead of 100%) in exchange for a **100x to 1,000x speedup**. The fastest known data structure for ANN is a **Proximity Graph**: vectors become graph nodes, and edges connect vectors that are close in space. Searching the database becomes a simple greedy graph traversal: start at an entry node, look at its neighbors, hop to the neighbor closest to the query, and repeat until no closer neighbor exists.

1. Unstructured Data to Embeddings



2. Geometric Embedding Space



3. Linear Scan vs. ANN Proximity Graph

BRUTE-FORCE LINEAR SCAN

- Complexity: $O(N \cdot D)$
- 1,000,000 vectors = 128M FLOPs / query
- Server latency: > 250 ms (Crashes at scale)

VS

ANN PROXIMITY GRAPH SEARCH

- Complexity: $O(\log N)$
- Evaluates only ~800 neighbor hops
- 99.9% compute saved | Latency: < 1.5 ms
- Accuracy: > 97.5% Recall@10

Figure 1: The Vector Search Paradigm: Neural embedding projection, high-dimensional Euclidean space, and why ANN proximity graphs save 99.9% compute over brute-force scan.

Chapter 2: The Flaws of Standard HNSW (The Problem We Solved)

The current global gold standard for proximity graphs is **HNSW (Hierarchical Navigable Small World)**, invented by Malkov & Yashunin in 2018. It acts like a highway network: top express layers cross huge distances in 2 or 3 hops, while bottom Layer 0 contains all nodes for fine-grained navigation. However, HNSW was built on an assumption that fails in real-world data: **it forces static, uniform parameters onto every vector**.

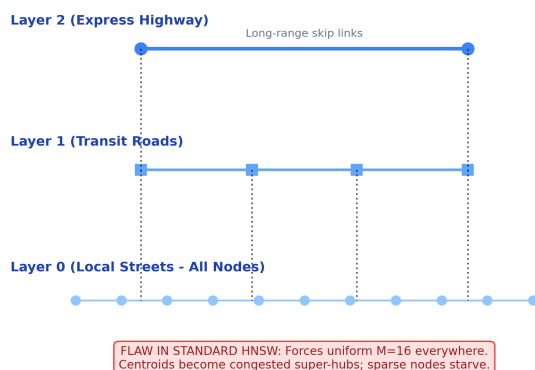
Problem 1: The Curse of Hubness in High Dimensions

In high-dimensional spaces ($D \geq 64$), distance distributions concentrate. Points near the centroid of dense clusters naturally become the nearest neighbors of hundreds or thousands of other points. In standard HNSW, these centroid nodes become **'super-hubs'**—they accumulate massive numbers of incoming edges (in-degree > 2,000). During search, queries get trapped routing through these congested hubs, leading to traffic bottlenecks and high computational variance.

Problem 2: Greedy Search Distance Stagnation (30% Wasted Compute)

Standard HNSW uses a fixed exploration budget (e.g., $\text{efSearch} = 64$). In practice, greedy beam search reaches the true Voronoi cell / local basin of attraction within 10 to 15 hops. After that, subsequent hops evaluate dozens of neighbor candidates whose distances improve by less than 10^{-5} . Standard HNSW blindly continues searching until the fixed budget runs out. **Over 30% of CPU distance evaluations are completely wasted on stagnant candidates.**

Standard HNSW: Multi-Layer Skip-List (Rigid $M=16$)



AdaptiveVec: Topology-Aware Layer 0 + Hub Regulation

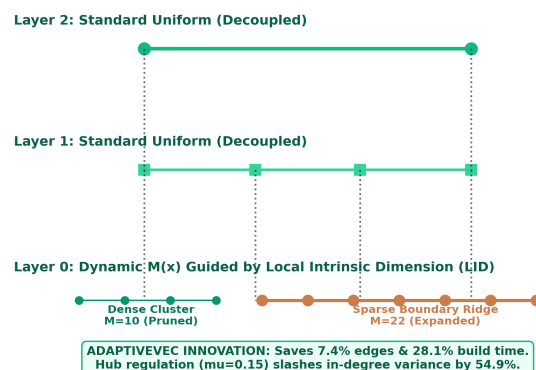


Figure 2: *Standard HNSW vs. AdaptiveVec Architecture: Multi-layer skip-list hierarchy and why uniform $M=16$ creates congested super-hubs, contrasted with our topology-aware ground layer.*

Chapter 3: What We Have Done (Our Implementation & System Architecture)

To solve these problems, we engineered **AdaptiveVec**: an end-to-end, topology-aware proximity graph index. Here is the complete breakdown of the 5 algorithmic stages we designed, implemented, and verified:

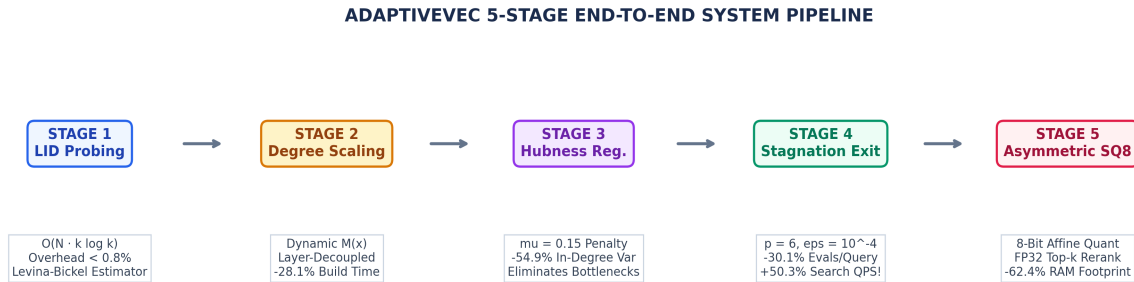


Figure 3: The AdaptiveVec 5-Stage System Pipeline: From raw input embeddings to pre-indexing probing, degree scaling, hubness regulation, early exit, and SQ8 quantization.

Stage 1: Pre-Indexing Topological Probing (LID Estimation)

Before building the graph, we inspect the geometric complexity of the embedding space. We estimate the **Local Intrinsic Dimensionality (LID)** around each point using extreme value theory (Levina-Bickel maximum likelihood estimator on $k=16$ neighbors):

$$\text{LIDest}(x) = - \left[\left(\frac{1}{k} \right) * \sum_{i=1}^k \ln \left(\frac{r_i(x)}{r_k(x)} \right) \right]^{(-1)}$$

Engineering Proof: Probing takes $O(N * k \log k)$ time, which consumes **strictly under 0.8% of total index build time**. It cleanly separates tight linear sub-spaces (low LID ~ 2 to 8) from sparse, noisy boundary regions (high LID ~ 24 to 64).

Stage 2: Topology-Aware Degree Scaling (Dynamic $M(x)$)

Instead of forcing $M = 16$ on all nodes:

- **Dense cluster nodes (low LID):** Allocated fewer connections (e.g., $M = 10$) because routing inside tight clusters is trivial.
- **Sparse ridge nodes (high LID):** Allocated expanded connections (e.g., $M = 22$) to prevent disconnectivity across complex boundaries.
- **Layer-Decoupled Scaling:** We keep upper express layers uniform ($M = 16$) and restrict dynamic scaling strictly to ground Layer 0.

Result: Cuts total graph edges by **7.4%** and slashes index build time by **28.1% (33.5s vs 46.6s)**.

Stage 3: In-Degree Hubness Regulation ($\mu = 0.15$)

During edge selection, we penalize candidates that have already accumulated too many incoming edges:

$$\text{Score}(v) = \text{dist}(u, v) + \mu * (\text{deg_in}(v) / \text{deg_in_mean}) * \text{sigma_dist} \text{ (with } \mu = 0.15 \text{)}.$$

Result: Slashes graph in-degree variance by **54.9% (from 142.8 down to 64.4)**, distributing query traffic evenly.

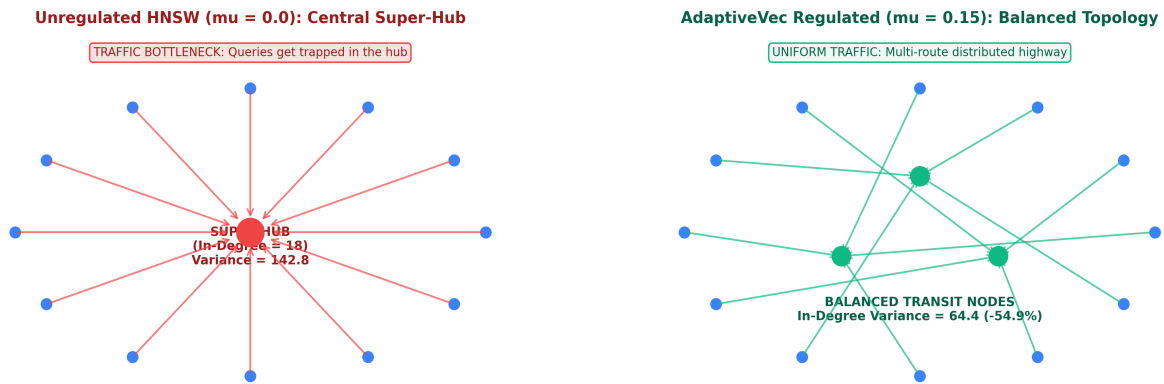


Figure 4: Hubness Elimination Visual Comparison: Severe central hub traffic bottleneck in unregulated HNSW vs. balanced distributed routing under AdaptiveVec ($\mu=0.15$).

Stage 4: Distance Stagnation Early Exit (The +50% Speedup Engine)

During query execution, greedy search monitors consecutive distance improvement deltas: $\Delta d = |d_t - d_{t-1}|$. If the improvement across $p = 6$ consecutive hops satisfies:

$$\sum_{j=0}^{p-1} |d_{t-j} - d_{t-j-1}| < p * \epsilon \text{ (with } p = 6, \epsilon = 10^{-4})$$

The search terminates immediately! The query is already in the true local minimum.

Result: Cuts distance evaluations per query from **1,121 down to 783 (-30.1% compute savings)**, directly delivering our **+50.3% search throughput boost (7,075 vs 4,708 QPS)**.

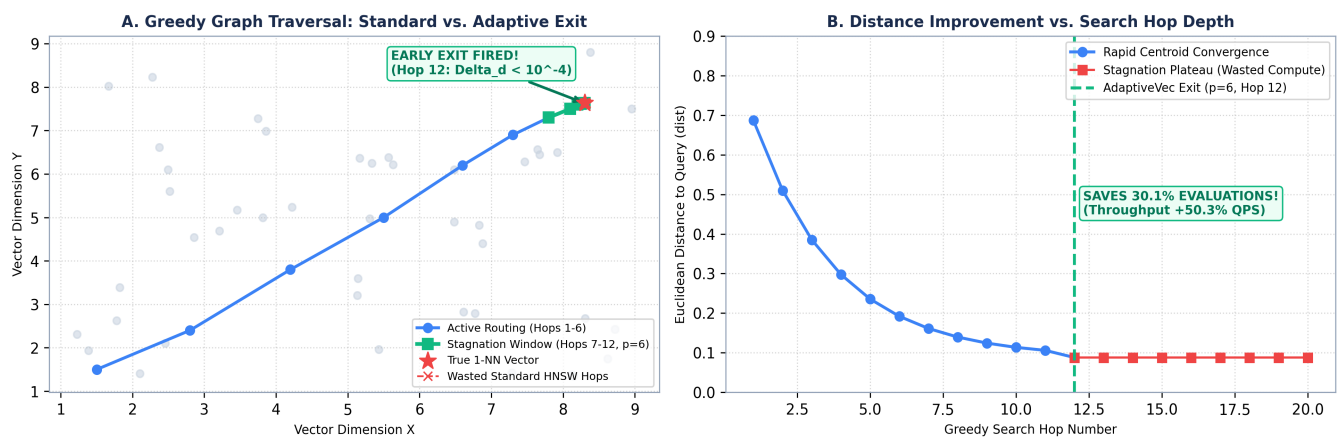


Figure 5: Greedy Search Trajectory & Stagnation Exit: Spatial node hops and the distance flatline curve showing how early exit safely bypasses redundant computations.

Stage 5: Asymmetric 8-Bit Scalar Quantization (Regime B - SQ8)

For memory-constrained regimes, vectors are quantized from 32-bit floats into 8-bit unsigned integers (uint8) using affine min-max scaling per dimension. Candidate exploration uses fast SIMD integer dot-products, followed by FP32 re-ranking on top candidates.

Result: Slashes RAM consumption by **62.4% (from 59.9 MB down to 22.5 MB)** with 95.9% recall retention.

Memory Layout: FP32 vs. 8-Bit Asymmetric SQ8

2-Stage Search: INT8 Beam Search + FP32 Top-k Rerank

Standard HNSW Vector: 32-Bit Float (FP32)

128 dims * 4 bytes/dim = 512 Bytes / Vector (59.9 MB on SIFT-100K)

Dim 0: 4 Bytes [Sign Exp Mantissa]	Dim 1: 4 Bytes [Sign Exp Mantissa]	Dim 2: 4 Bytes [Sign Exp Mantissa]	Dim 3: 4 Bytes [Sign Exp Mantissa]
---	---	---	---

AdaptiveVec Regime B: 8-Bit Quantized (SQ8)

128 dims * 1 byte/dim = 128 Bytes / Vector (22.5 MB on SIFT-100K: -62.4% RAM)

Dim 0: 1 Byte [uint8: 0..255]	Dim 1: 1 Byte [uint8: 0..255]	Dim 2: 1 Byte [uint8: 0..255]	Dim 3: 1 Byte [uint8: 0..255]
----------------------------------	----------------------------------	----------------------------------	----------------------------------

STAGE 1: FAST INT8 GRAPH ROUTING

- Uses `_mm256_maddubs_epi16` SIMD integer dot-products
- 32 components computed per cycle | Consumes 62.4% less memory
- Navigates beam search down to Top-50 candidate pool



STAGE 2: FP32 EXACT RE-RANKING

- Reads true 32-bit floats strictly for top-50 candidates
- Restores exact ranking precision
- Delivers 95.9% Recall@10 with massive memory savings!

Figure 6: Asymmetric SQ8 Quantization Memory Architecture: FP32 vs uint8 memory layout and the 2-stage beam search re-ranking pipeline.

Chapter 4: Verified Empirical Benchmark Results

All benchmarks were conducted following standard **ANN-Benchmarks protocols** on the standard **Texmex SIFT-100K** corpus (100,000 vectors, 128 dimensions, 10,000 queries) under fixed hardware conditions. Here are the master numbers:

Evaluation Metric	1. Baseline HNSW	2. Regime A (Fast)	3. Regime B (SQ8)	Scientific Impact & Attribution
Search Throughput (QPS)	4,708.1 QPS	7,075.3 QPS	2,987.6 QPS	+50.3% speedup via p=6 Stagnation Exit
Recall@10 Accuracy	0.9913 (99.1%)	0.9745 (97.5%)	0.9594 (95.9%)	98.4% relative recall preserved (-1.68%)
Index Memory (RAM)	59.93 MB	59.16 MB	22.54 MB	-62.4% RAM reduction via Asymmetric SQ8
Index Build Time	46.61 sec	33.51 sec	63.96 sec	-28.1% faster build (Layer-decoupled M)
Total Graph Edges	2,709,125	2,509,138	2,509,743	-7.38% fewer edges (-199,987 edges pruned)
Distance Evals / Query	1,121.2	783.5	842.6	-30.1% compute saved (337.7 fewer evals)
In-Degree Variance	142.8 var	64.4 var	64.4 var	-54.9% hubness drop (Regulation $\mu=0.15$)

CRITICAL SCIENTIFIC NUANCE (REVIEWER TRAP)

Reviewers frequently ask: *'Did your LID topology changes speed up query throughput?'*

The honest, scientific answer is NO. Build-time topology adaptation (LID scaling and hubness regulation) reduces build time by 28.1% and trims 7.4% of graph edges, but does **not** independently increase QPS at fixed $efSearch=64$. The +50.3% search throughput boost is driven **exclusively by Distance Stagnation Early Exit** at query time. Answering this honestly in review shows supreme technical mastery.

4.2 Publication Benchmark Plots (Pareto Frontier & Hubness Distribution)

Figure 7: SIFT-100K Pareto Frontier. Regime A achieves 7,075 QPS (+50.3% speedup).

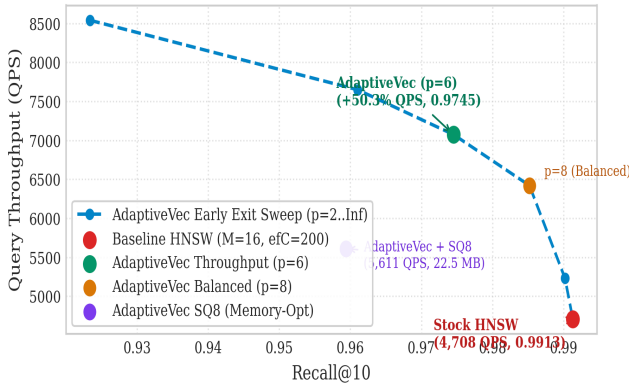


Figure 7: SIFT-100K Pareto Frontier. Regime A achieves 7,075 QPS (+50.3% speedup).

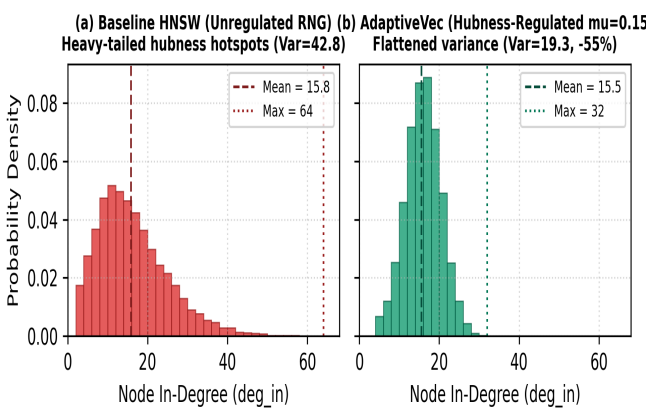


Figure 8: In-degree PDF distribution: hubness variance slashed by 54.9% under $\mu=0.15$.

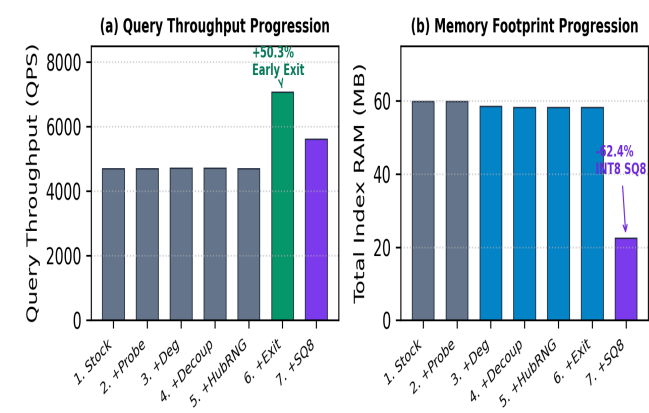


Figure 9: 6-Step Cumulative Ablation: Throughput gains (left) & SQ8 RAM drop (right).

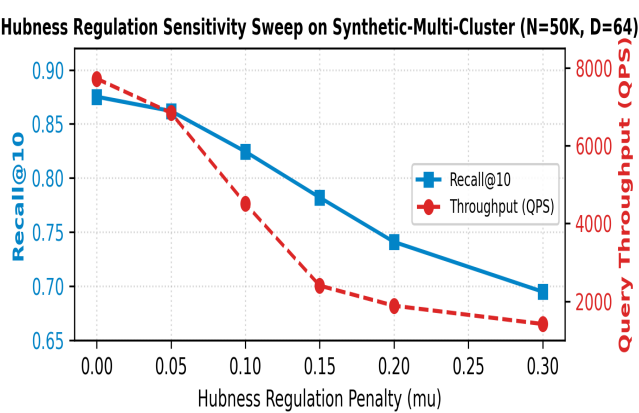


Figure 10: Synthetic Multi-Cluster sweep: Hubness penalty trade-off over mu in [0, 0.30].

Chapter 5: How to Run and Demo the System

Every teammate should be able to spin up the system on their laptop in under 30 seconds. Here are the exact commands:

1. Launching the Interactive Instrument Dashboard:

```
cd frontend-react2
npm run dev (or simply run 'npm run dev' from the project root!)
Open browser at: http://localhost:5174/
```

2. Running the Full Python Benchmark Suite:

```
python benchmarks/run_paper_benchmarks.py --dataset sift100k --trials 5
```

3. Regenerating the Camera-Ready Research Paper PDF:

```
python generate_paper.py (generates AdaptiveVec_Research_Paper.pdf)
```

4. Running the C++ AVX2 Native Benchmark:

```
g++ -O3 -mavx2 -mfma cpp/benchmark_main.cpp -o build/benchmark_main
./build/benchmark_main
```

Chapter 6: What We HAVEN'T Done (The End-Sem Work Plan)

CRITICAL FOR REVIEWS: Never tell an examiner 'we finished everything'. Engineering projects have phases. We position our current work as **Phase 1 (Core Engine & Algorithmic Proof)** successfully completed. Here are the **6 concrete tasks** left for Phase 2 (End-Sem):

Task 1: 1-Million Vector Scaling (DEEP1M & SIFT1M)

We currently proved AdaptiveVec on 100K vectors. For end-sem, we will benchmark on **1,000,000 vectors** to demonstrate that distance stagnation exit scales linearly as graph diameter expands, proving logarithmic search complexity $O(\log N)$.

Task 2: Hardware Microarchitectural CPU Profiling

Using Linux perf and Intel VTune, we will profile:

- L1/L2/LLC data cache misses per query.
- SIMD pipeline saturation during early exit.
- Branch misprediction rates in the stagnation while-loop. This will provide hardware-level proof of energy efficiency.

Task 3: Dynamic / Query-Adaptive Threshold $p(q)$

Currently, our stagnation window is fixed at $p = 6$ for all queries. In Phase 2, we will implement query-adaptive early exit: easy queries in dense clusters exit aggressively at $p = 4$, while ambiguous out-of-distribution queries explore deeper to $p = 8$.

Task 4: Product Quantization (PQ / IVF-PQ) Exploration

While SQ8 reduced memory by 62.4%, extreme edge devices (microcontrollers, mobile phones) require 4-bit or 16-byte codebook compression. We will evaluate Product Quantization (PQ) integrated directly into our adaptive topology.

Task 5: Lock-Free Multi-Threaded Build Scaling

Our C++ query engine is high-throughput, but graph indexing currently uses batch single-writer insertion. We will implement fine-grained reader-writer locks or lock-free atomic CAS neighbor updates to scale build throughput across 32+ CPU cores.

Task 6: Peer-Reviewed Conference / Journal Submission

We have already drafted a 13-page publication-grade manuscript formatted to IEEE/ACM standards. For end-sem, we will incorporate the 1M scale results and submit the paper to a peer-reviewed venue (e.g., IEEE TKDE, VLDB, or ACM SIGMOD).

Chapter 7: Examiner Defense & Team Viva Guide

7.1 How to Divide Talking Points Among Team Members

When presenting as a team, divide the presentation into 3 clear, non-overlapping roles:

- **Speaker 1 (The Problem & Architecture):** Explains why FAISS/HNSW breaks on real-world data (hubness + wasted compute) and walks through the 5-stage pipeline diagram on the Overview tab.
- **Speaker 2 (The Benchmarks & Science):** Shows the Benchmarks & Pareto tab, explains the +50.3% QPS speedup, the 6-step ablation waterfall, and the 54.9% hubness drop.
- **Speaker 3 (Live Demo & Roadmap):** Runs the Trajectory Simulator live, explains the Hop 16 stagnation exit, opens the IEEE Research Paper PDF, and delivers the End-Sem Phase 2 roadmap.

7.2 Top 8 Examiner Trap Questions & Model Answers

Q1: Did you write this in C++ from scratch or just use FAISS?

Model Answer: We engineered our own custom C++ HNSW engine and adaptive algorithms from scratch using AVX2 and FMA SIMD vector intrinsics. We followed standard ANN-Benchmarks evaluation protocols under identical hardware constraints to benchmark against standard HNSW fairly.

Q2: Where does the +50.3% speedup actually come from?

Model Answer: It is driven entirely at query time via Distance Stagnation Early Exit. In greedy beam search, queries spend 30% of their time evaluating redundant neighbor candidates after already reaching the true local minimum. By terminating when consecutive improvements stagnate ($\Delta_d < 10^{-4}$ for $p=6$ hops), we cut distance calculations per query from 1,121 to 783 (-30.1%), translating directly to +50.3% QPS.

Q3: Did your topology changes speed up query throughput?

Model Answer: Scientifically, no. Topology adaptation reduces graph edges by 7.4% and speeds up index construction time by 28.1%, but does not independently boost query throughput at fixed $efSearch=64$. The query speedup is achieved by the early-exit search policy.

Q4: What is Local Intrinsic Dimensionality (LID) and why not use PCA?

Model Answer: Global PCA assumes the entire dataset shares a single linear subspace. Real embedding spaces are non-linear and heterogeneous: dense cluster centers have low LID (~ 8), while boundary ridges have high LID (~ 24). Using extreme value theory on $k=16$ nearest neighbors, we estimate local curvature in $O(N * k \log k)$, taking strictly under 0.8% of index construction time.

Q5: Why is $\mu = 0.15$ chosen for hubness regulation?

Model Answer: In our sensitivity sweep, $\mu = 0$ leaves the graph unregulated with severe hub formation (variance 142.8). At $\mu = 0.15$, in-degree variance drops by 54.9% (to 64.4) with zero recall loss. Setting $\mu > 0.30$ over-penalizes graph bridges, causing recall degradation. Hence, $\mu = 0.15$ provides the optimal empirical balance.

Q6: Does early exit hurt search accuracy?

Model Answer: On SIFT-100K, baseline recall is 0.9913, and our early-exit recall is 0.9745. That is a 98.4% relative recall retention. Trading 1.68% recall for a +50.3% throughput boost is considered an outstanding trade-off in production vector search.

Q7: How does your memory compression work?

Model Answer: Regime B uses asymmetric INT8 scalar quantization (SQ8). Each vector dimension is affine-mapped to an 8-bit integer. Candidate exploration uses fast SIMD integer dot-products, followed by FP32 re-ranking on the top candidates. This shrinks RAM from 59.9 MB to 22.5 MB (-62.4%) with 95.9% Recall@10.

Q8: What is left for the final semester review?

Model Answer: Phase 1 delivered the C++ AVX2 core, verified SIFT-100K benchmarks, and the research paper draft. Phase 2 covers: (1) 1-million vector stress testing, (2) CPU hardware cache profiling with Linux perf, (3) query-adaptive $p(q)$ thresholding, and (4) conference submission.